“All I Wanted was a Banana”

Programming part 9: The worst decisions in programming

Dr Jo

2026-10-11



Gorilla Eating—image from Wikimedia Commons

> "I think the lack of reusability comes in object-oriented languages, not functional languages. Because the problem with object-oriented languages is they've got all this implicit environment that they carry around with them. You wanted a banana but what you got was a gorilla holding the banana and the entire jungle.
> — Joe Armstrong

We continue our fun-filled exploration of the exotic computer language Zyzyxia. So far we've swiped from LISP,⌘ J⌘ and Forth⌘ and introduced POWER notation⌘; our next target is Erlang.

I recently lamented the clunky programming that bedevils medicine⌘—and contrasted this with the legendary 99.9999999% uptime of the million+ lines of Erlang code that powers the Ericsson AXD301 ATM switching system.[1]

This focus of this post is not, however, either a paean of praise for Erlang, nor some sort of beat-up on object-oriented programming. Well, not completely. I am more interested in what we can learn. Let's start with some history.

Axing AXE-N

Ericsson makes telephone exchanges. These have a variety of demanding requirements, including (a) not breaking; (b) tolerating faults without breaking; (c) 'soft real-time' i.e. not breaking if something is a bit delayed; (d) running non-stop without breaking; and (e) the ability to change an entire program while it is still running, without breaking.

By pretty much any measure, the Ellemtel AXE-N project was a huge failure. On their website, Ericsson describes how L.M. Ericsson and Televerket set up Ellemtel as a successful joint venture in the 1960s. They came up with the successful AXE electronic switching system in the 1970s. AXE-N was to be its successor, a single, universal, global telecommunications system.

At least, in theory. Asynchronous Transfer Mode (ATM[2]) tech was to be used to build something remarkably similar to the Internet, but for all telecoms. The project started in 1987, and SEK 10

[1] As I pointed out there, Armstrong is a bit more muted in his PhD thesis about those 'nine nines'. So yes, perhaps a bit of legend here :)

[2] Don't confuse this with automated teller machines.

billion later,[3] died in ignominy in December 1995.

The key change here was a move from Ericsson's own in-house PLEX programming language to the "industrial standard programming language" C++. The problem with complex projects that die is that they are complex, and so it's easy to find a thousand reasons for failure. The big change to an "industrial standard" can be rationalised as "not the real problem". We all know that complex projects fail for other reasons—overreach, bad management, poor synchronisation of the moving parts, and excessive complexity itself. A scapegoat can be found.

Such failures are less forgivable when the real defect can be found and replaced, with subsequent astounding success—and the error is then repeated! First, let's look at the fix.

Erlang

There are two complementary origins for the name 'Erlang'. It's a convenient portmanteau of the initial syllables of 'Ericsson' and 'Language'; but it also refers to the Danish telecommunications pioneer Agner Krarup Erlang, who invented queueing theory.

Fortunately for Ericsson, when AXE-N failed, Erlang had been waiting in the wings, under development for a decade—in house—by Joe Armstrong and a handful of other engineers. Erlang is a functional language. The fundamental idea behind a 'functional language' is that it is made up of 'pure functions'. These are functions because they take zero or more pieces of information ('arguments') and produce a single result. They are pure because they produce no side effects. For telecommunications, this is a very good thing. Erlang has other good things.

Erlang is designed for telecoms. Its key features are (a) not breaking; (b) tolerating faults without breaking; (c) 'soft real-time' i.e. not breaking if something is a bit delayed; (d) running nonstop without breaking; and (e) the ability to change an entire program while it is still running, without breaking.

Over the next 3 years, Ericsson invested a lot of face-saving resources into repairing the AXE-N debacle. The Erlang team was expanded from 2 to over 60 people, who commandeered the

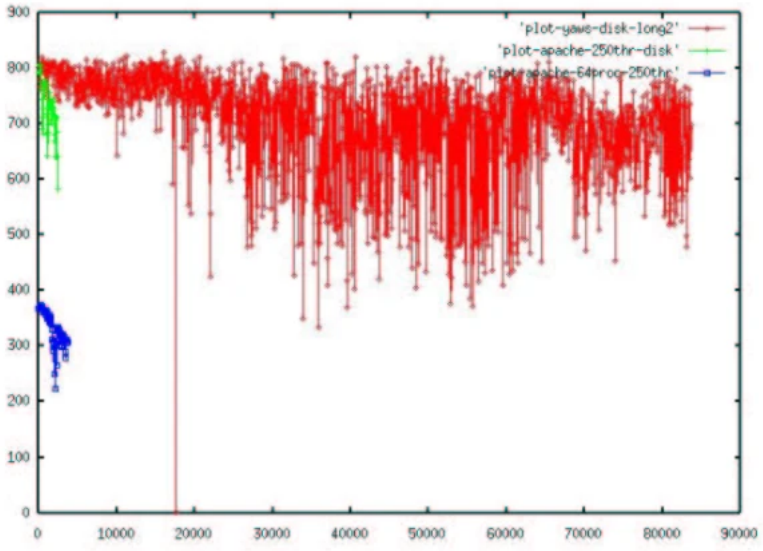
[3] North of a billion USD, one of the largest failed telecom projects ever, in any currency.

existing hardware and by 1998 produced and then released the AXD301 ATM switch—with the aforementioned, legendary performance.

And now for the fun bit. Also in 1998, Ericsson forbade the use of Erlang in future projects (favouring C++). In December 1998, Erlang was open-sourced and most of the Erlang team resigned. Subsequently ...

In 1999, management at Erlang suddenly stopped having 'problems'. These were replaced by 'issues'. Issues then rapidly transformed into 'challenges', which in turn gave way to 'opportunities'. Their opportunities became so great that in 2004, Ericsson re-hired Armstrong and—it is said—implemented a "don't ask permission, just use it" approach to Erlang.

One would hope they paid him well.



Web server running Apache vs Yaws (Erlang). See text.

What can we learn?

Consider the above graphic from 2002. At the top, in red, we have success. An Erlang-based web server (Yaws) sustains communication at around 800 KB/second with 80,000 interrupting processes, running on NFS (the Network File System). On the left in blue, the more conventional Apache runs at half that communication rate, and in this same environment, chokes up and dies with about 4000 interrupting processes. The green is running on the local machine, rather than NFS. It also dies soon.

Armstrong calls this 'Concurrency-oriented programming'. Everything is simply designed around communicating processes. Built into the design are small, light, independent processes that deliberately don't have side effects. All interaction is through asynchronous messages.

Different readers will take away different lessons. Some might reasonably conclude "If you need fault-tolerant concurrency, favour Erlang over C++ in many circumstances". I can however see some more general lessons. These relate to:

1. The worst decision in programming—and the WORM that turns.
2. Globals.
3. Local variables.
4. OOPS!
5. Failure.

Let's take each in turn.

1 The worst decision

There are multiple candidates for the worst idea in computer programming. GOTO statements that invite "spaghetti programming". Tight coupling between modules, or between the information layer and the functional layer.⌘ Failing to embrace literate programming.⌘ Insightless use of NoSQL.⌘ Excessive code complexity. But here's an example of my #1 for defective code:

```
N = N + 1;
```

I'm sure at least some readers are thinking "C'mon, why's that so bad then? All we have here is a box (called 'N') containing a value, and all we're doing is adding one. This is standard coding. What could possibly be so bad about this?" Some programmers will be even more forthright.

Consider instead a language where a 'variable' can only ever be written once: it's either undefined, or it has a constant value. For me, this sort of decision about the 'flavour' of a language is fundamental, important and profound.

From now on, let's refer to such single-assignment variables as "WORM"—Write Once, Read Many. You'll meet this in Erlang, Rust and ***Zyzyxia***—and then squirrelled away in many other languages.[4] For example, in modern JavaScript, we have the option of `const N = 123;` — but it still supports `var`, embraces `let` — and `const` objects can be modified!

WORM turns you. It changes the entire way you program. It constrains the way you use state. The first and most obvious impact is on globals.

2 Globals

Good programmers rightly disparage globals that are mutable. There are two distinct issues here. The first relates to one of those big problems in all programs—naming. If you choose names badly, nothing will help you. This is more a social problem than a programming one. If you are using double-precision floating point numbers, then setting `Pi` to anything other than `3.141592653589793~` is just silly.

The second problem comes when some well-meaning person[5] re-defines a global value. This is still a social problem related to communicating numerical values, but it rapidly becomes a practical problem of things failing, sometimes rather intermittently.

WORM mitigates the first problem and eliminates the second. If I say …

```
SET <Phi> 1.618033988749895~
SET <Hello> <Hello, world!>
```

[4] Although Rust buggers things up by (of course) adding a 'mut' option.
[5] Or idiot.

... in my language ***Zyzyxia***, then these values are immutable. In fact, I can't even re-define a verb once I've SET its value. And with this approach, globals lose all of their terror.

By making each global immutable, you do three things. You decrease complexity, because there are just two options: undefined, or that constant value. You substantially decrease the problem of mutable state. But most important, you alter how people think about programming. Let's try an example.

3 Local variables

To calculate the period of a pendulum accurately, it's useful to be able to work out the arithmetic-geometric mean or agm. That's because a pretty precise period for a pendulum is given by this equation:

$$T \approx 2\pi\sqrt{\frac{L}{g}}\frac{1}{\text{agm}\left(1, cos\left(\frac{\theta}{2}\right)\right)}$$

How do we calculate the agm()? Here's some code written in C:

```
#include <math.h>
  // Return arithmetic-geometric mean (positive arguments).
double agm (double a, double g)
{
if (a <= 0 || g <= 0) { // domain error
return 0;
}
for (;;) {
  double a0 = a;
  double g0 = g;
  a = (a0 + g0) / 2;
  g = sqrt(a0 * g0);
  if (a == g || fabs(a - g) >= fabs(a0 - g0)) {
    return a;
    }
  }
}
```

The idea is that we repeatedly loop around until the average of the arithmetic and geometric mean converges on the answer. The code is crude but effective, converging rapidly.[6] And yes, it uses that N=N+1 paradigm.

[6] If you pop it into Claude, the bot will correctly point out several defects—it may misbehave with infinite values and those not-a-number (NAN); the return value of 0 is ambiguous; the test for convergence is subtle.

Here's something similar, written in ***Zyzyxia***:

```
SET <agm>
< agm (aMean gMean)
  { RETURN aMean
    | } CASE NEARLY aMean gMean
  SET [.gMean.] geometric_mean
  SET [.aMean.] mean COPY
>
```

You can get the gist of the code, simply reading left-to-right—even though POWER parses right to left.[⌘] First, we can see that `agm (aMean gMean)` recurses, calling itself. It even provides a concise hint of how it's used, taking an arithmetic and geometric mean as arguments.

We soon see the termination condition: if the two are `NEARLY` equal, we return one of them.[7] Note how the parochial[8] values `aMean` and `gMean` are respectively set to the arithmetic and geometric means, courtesy of the corresponding verbs.

Finally, on the right, we can work out that `COPY` copies the submitted list from the stack, thus providing a copy for both `mean` and `geometric_mean` to work on. A very different approach, conditioned by the fact that variable re-allocation is forbidden—and the stack is there to use. Little constraints have big consequences.[9]

4 OOPS!

As Armstrong points out, the fundamental difference between functional languages and Object-oriented programming (OOP) is that functional code avoids side effects and 'mutable state'. If you further constrain things like, well, all variables, this cleans up not just your code space but your mental coding space. Or, as Armstrong says:

[7] It's awfully easy to muck up the comparison of two floats, so solid code to do this is wrapped up in the `NEARLY` command—rather than relying on empiric guessing code.

[8] 'Local variables' in ***Zyzyxia*** are 'parochials'. It's a parochial thing :)

[9] For those who are interested, let's step through agm. POWER interprets from the right—and we mean really on the right, the end of all the code, where we pick up a value from the stack and makes a `COPY`. This copy is then submitted to `mean`; the residual value on the stack goes to `geometric_mean`. If the resulting values in `aMean` and `gMean` are `NEARLY` the same, we return one of them; otherwise we just recur. The parenthesis around `(aMean gMean)` tells us we're dealing with a list of two values, so we know that we apply the verb like this: `agm (1~ 2~)`

> If you have referentially transparent code, if you have pure functions — all the data comes in its input arguments and everything goes out and leave no state behind — it's incredibly reusable."

I think it was in 1987 that I watched OOP take off. One thing that bemused me about the spread of OOP at the time was the near-complete lack of evidence of superiority.[10] If OOP were a drug, I don't think it would get past even the currently hugely compromised FDA.

OOP clearly has a powerful place where (a) the hierarchical structure of objects closely maps to the structure of the space you're trying to model. Unfortunately, it may be a suboptimal fit for (b) everything else, especially because the natural tendency in OOP is to spread your mutable state around, throughout your program.

Over the years, I've looked for decent studies comparing e.g. OOP and functional programming. You can find a few. Many of these are 'toy' comparisons. For example back in the 1990s, Rachel Harrison et al. compared C++ and SML ('Standard ML' that is mainly functional), against twelve sets of algorithms, and found more errors with SML.[11] The remaining ones are subject to controversy. For example, in 2014 Baishakhi Ray and colleagues pulled 729 projects from GitHub (80 million lines of code, 29k authors) and tried to tease out the effects of language on software quality. They found several effects—benefits of strong typing (which we'll cover in a later post), and a modest benefit of using functional languages, but the main effects they teased out related to other factors such as project size and team size.

The water is even more muddy today, because most popular languages boast "multi-paradigm" capabilities. You can use objects, functional, and often other options too. Sadly, this cuts to the heart of the problem—excessive complexity.

Complexity is still the enemy. With OOP, often "All I wanted was a banana".

---

[10] Or even equivalence, when it comes to things like code re-use.

[11] There's more here: https://scholar.google.com/citations?user=BqWM2XwAAAAJ&hl=en

5 Failure

I think the stand-out difference for Erlang is not write-once 'variables' or better management of things like mutexes (effectively, by eliminating them). Even the possible benefits of a functional approach are exceeded by something else. It's Armstrong's approach to error. Here's a quote from his PhD thesis, once more:

> The software systems in the case study are so reliable that the people operating these system are inclined to think that they are error-free. This is not the case, indeed software errors did occur at run-time but these errors were quickly corrected so that nobody ever noticed that the errors had occurred.

There are two main ways we can approach software errors: terror or accommodation. The traditional approach is that the program clutches its pearls, cries out "Woe is me!" and dies catastrophically.[12] This is usually less than useful.

An alternative is simply to acknowledge that errors will occur, and to plan for and accommodate them. Unless code can be proven not to fail (under the required circumstances, which may be very broad), then it is more reasonable to plan for, trap and manage all sorts of errors, rather than to simply hope.

Erlang's philosophy here is to have supervisor modules that are continually waiting for something to fail, and to rip all or almost all defensive error code out of the working code in those supervised modules. But even if we're not going there, we can still respect, honour and plan for error.

For example, ***Zyzyxia*** has this[13] construct:

```
{ do_default | Yes }
{ do_this | EQ ConditionFoo }
{ do_that | EQ ConditionBar } TRAP { do_something_here } TRY
```

As usual, we parse strictly right-to-left using POWER.

[12] This may even be mandatory.

[13] The example is simply illustrative and deliberately a bit vague. When we get to CASE statements, we'll see the structure is very similar; already you can work out that ConditionFoo and ConditionBar are Nouns, and spot all of the lowercase verbs. For various reasons, I haven't fully embraced the Erlang take on things.

In summary

Complexity is still the enemy. However, we have tools that don't only help us to manage it, but also subtly change how we see state, local and global variables, and where errors fit in. I've tried to build some of this into ***Zyzyxia***.

This is as much a cultural change as it is about the programming language, but sometimes language can influence how we think, and then how we behave.

➳ In my next post, I'll look at strong typing—and take it to ridiculous excesses. We'll also learn real respect for the stack.

My 2c, Dr Jo.

⌘ This symbol is used to indicate posts where I've discussed the flagged topic in more detail.

Copyright © 2026 JM van Schalkwyk

This work is licensed under the Zyzyxia Public Licence, version 1. You may convey it only together with its Corresponding Literate Source Document, as that Licence requires. All rights are reserved in respect of AI Training Use, including for the purposes of any text-and-data-mining exception or opt-out provision under applicable law. Submitting this work as input to a Generative AI System is prohibited.